FR

Bienvenue sur Sakata

Ce portail est un sanctuaire numérique dédié à la préservation et à la transmission de l'identité culturelle du peuple **Sakata**.

Notre objectif est d'unir nos racines ancestrales avec la splendeur numérique moderne, en offrant à la fois des archives vivantes, des sagesses intemporelles et un forum d'échange intergénérationnel.

Suivant →

> *Une PWA n'est pas une appli mobile « light ». C'est la reconnaissance qu'en 2026, la frontière entre web et mobile est devenue artificielle, et que les utilisateurs africains méritent autant la qualité d'expérience d'une vraie appli que ses contraintes (offline, push, install).*

JUSTIFICATION

Pourquoi maintenant ?

Le contexte africain

- **70% du trafic africain est mobile**, principalement sur Android entry-level (RAM 2 Go, processeurs lents).
- Connexions **3G majoritaires** en zones rurales, latences > 500ms fréquentes.
- Coût des données : 1 Go peut représenter une journée de salaire — l'offline est un argument économique.
- Play Store : payer 25$ pour publier, plus le coût de maintenance d'une app native… La PWA contourne ce coût.

Le contexte diaspora

- La diaspora est sur iOS Belgique/France/Canada (RAM 4 Go+, fibre).
- iOS supporte les PWA installées (depuis iOS 16.4) — install à l'écran d'accueil, push notifications.
- Le mobile est le device principal de consommation culturelle (audio, vidéo, lecture).

CONCLUSION

Une PWA bien faite couvre 95% des besoins mobile à 5% du coût d'une app native.

DÉCISIONS TECHNIQUES

Choix du stack

Service Worker : next-pwa ou Workbox direct ?

OPTION	POUR	CONTRE	VERDICT
next-pwa (wrapper)	Configuration déclarative, intégration Next.js	Maintenance lente (dernière release > 1 an), pas Next 15+	❌ Obsolète
@serwist/next	Successeur officiel de next-pwa, maintenu, Next 15 ✓	Plus jeune, moins d'exemples	✅ Recommandé
Workbox direct + custom SW	Contrôle total, dernières features	Beaucoup de boilerplate, plus de maintenance	🟡 Si besoin spécifique

CHOIX

`@serwist/next` — c'est ce qu'utilisent maintenant les projets Next.js qui veulent une PWA propre.

Stratégie de cache

TYPE DE RESSOURCE	STRATÉGIE	JUSTIFICATION
HTML pages dynamiques	NetworkFirst (timeout 3s, fallback cache)	Contenu peut changer, mais offline = fallback acceptable
API GET /articles, /profiles	StaleWhileRevalidate (24h)	Articles changent rarement, fraîcheur en arrière-plan
Images Supabase Storage	CacheFirst (30 jours)	Immutables (URL avec ID), gain énorme
Mapbox tiles	CacheFirst (7 jours)	Tiles immutables, mais limite quota Mapbox
Polices, JS, CSS	CacheFirst (1 an, hash dans URL)	Build artifacts immutables
POST/PUT/DELETE	Pas de cache, network only	Mutations doivent être en ligne

Push notifications

Déjà partiellement implémenté (table `push_subscriptions`, API routes `/api/push/subscribe` et `/api/push/unsubscribe`). Il manque :

- L'envoi effectif de notifications (worker côté serveur via web-push).
- L'UX d'opt-in non intrusif (pas de popup au premier visit).
- La centralisation des préférences dans `/profil/notifications`.

EXÉCUTION

Plan d'implémentation — 3 sprints

Sprint 1 — Fondations PWA (1 semaine)

1. Installer `@serwist/next` et configurer `next.config.ts`.
2. Créer `public/manifest.json` avec couleurs Brume, icônes 192px et 512px.
3. Définir le `service-worker.ts` avec runtime caching configuré.
4. Ajouter `<link rel="manifest" />` et meta theme-color dans le layout.
5. Tester install banner Chrome Android + iOS Safari.
6. Validation Lighthouse PWA score > 90.

Sprint 2 — Offline & cache stratégique (1 semaine)

1. Implémenter une page `/offline` brandée Sakata.
2. Pré-cacher les routes critiques au build (homepage, /savoir, /chat).
3. Cache des images d'avatar et hero avec stratégie CacheFirst.
4. UI : indicateur visuel quand l'app est offline (banner discret).
5. Tests sur connexion 3G simulée (Chrome DevTools).

Sprint 3 — Push & install UX (1 semaine)

1. Worker Node.js avec `web-push` pour envoyer les notifs.
2. API `/api/push/send` appelable depuis le admin pour broadcast.
3. Triggers automatiques : nouveau message chat, nouvel article publié, mention forum.
4. Composant `InstallPrompt` qui apparaît après 3 visites (pas avant).
5. Préférences notifications par catégorie dans `/profil`.

À ANTICIPER

Pièges connus

IOS SPÉCIFIQUEMENT

- iOS Safari demande de cliquer le bouton de share puis « Sur l'écran d'accueil » — pas d'install banner natif comme Android.
- Les push notifications iOS nécessitent que la PWA soit **installée** (pas juste ouverte dans Safari).
- iOS limite le storage IndexedDB à ~50 MB — ne pas y stocker d'audio/vidéo.

CACHE SUPABASE REALTIME

Le service worker ne doit JAMAIS cacher les WebSockets Supabase. Exclure explicitement `realtime/v1/websocket` dans la config Serwist.

VERSIONING DE CACHE

Bumper la version du SW à chaque release majeure. Sinon, les utilisateurs restent bloqués sur l'ancienne version pendant des semaines.

COMMENT MESURER

Métriques de succès

MÉTRIQUE	CIBLE	OUTIL
Lighthouse PWA score	> 90	Chrome DevTools / CI
LCP P75 sur 3G simulé	< 2.5s	WebPageTest / RUM
Taux d'install (Android)	> 5% des MAU	navigator.getInstalledRelatedApps
Re-engagement via push	> 15% CTR	PostHog event tracking
Pages vues offline	> 200/mois	SW analytics personnalisé

BUDGET

Coût estimé

- **Temps dev :** 3 semaines × 1 dev = 0€ si interne, ~6 000€ si freelance.
- **Service push :** 0€ (web-push est gratuit, on s'auto-héberge).
- **Stockage cache :** 0€ (côté navigateur uniquement).
- **Total :** 0 à 6 000€ — l'option la moins chère pour étendre la portée mobile.

Sakata.com — Documentation interne · roadmap-pwa-mobile · v2026.4

« Moto oyo ayebi te kotombola miso na likolo, akoki te koyeba esika oyo mbula eutaka. » — Proverbe Sakata

SAKATA

© 2026 — Brume de la Rivière. Tous droits réservés.

CULTURE ECOLE LANGUE HISTOIRE FORUM